ISA5810 DM2026 Lab 1 — Master Notebook Questions

Fill in this file as you work through DM2026-Lab1-Master.ipynb. Submit this completed file converted to PDF together with the rest of your Lab 1 work.

Please do not change the style of this document.

Question numbers 1–10 follow the same order as the notebook (top to bottom).

Student Information

Please complete the fields below. For **English Name** and **Chinese Name**, you may fill in **only one** if you do not have the other.

- **English Name: Bui, Phan Tung**
- **Chinese Name:**
- **Email: tungbuiphan.cs15@nycu.edu.tw**
- **Student ID: 115550809**
- **University: NYCU**

Instructions

These questions go with the Master Jupyter notebook (DM2026-Lab1-Master.ipynb). Each question is tied to a specific notebook section. Open the notebook, find **Question N (take home)**, and write your answer here.

We **strongly** suggest running the notebook to learn the lab, but the TAs will **not** audit whether you executed every cell to answer the questions in this document.

How to answer each question

1. Find the **notebook section** listed under the question (use the Table of Contents if needed).
2. Read the task; change parameters when asked (e.g. stop_words, thresholds, k, minSup).
3. Write a clear answer in the **Answer** space below each question.
4. Leave the question text unchanged; write only in the answer and disclosure fields.

Sources, honesty, and AI use

Acceptable vs. Unacceptable AI Usage: Generative AI should serve as an interactive tutor to guide your understanding, not as an automated solver completing the assignment for you.

- **Permitted (AI as Guide & Editor):**
  - Asking AI to explain unfamiliar theoretical concepts, discuss algorithmic trade-offs, or troubleshoot runtime errors in the notebook.
  - Polishing the grammar, phrasing, and readability of your own original, self-written drafts.

- **Prohibited (AI as Solver):**
  - Copying assignment questions directly into an AI tool to generate answers.
  - Using AI to fabricate experimental results or observations without executing the code in DM2026-Lab1-Master.ipynb.
  - Submitting machine-generated prose wholesale.

- **External material:** If you use websites, books, papers, or documentation beyond the notebook, cite them with **clickable links** (or DOI/ISBN when no URL exists).

- **When AI was used:** Your answer is acceptable if you (1) **disclose** AI use and (2) include **at least one external link** that validates the information in your answer (official docs, textbook, paper, or reputable tutorial — not the AI chat alone as the only source).

- **When AI was not used:** You may write "No" under **AI used** or leave it blank only if you truly did not use AI for that question. References are still encouraged if you looked anything up.

Under each question, complete:

- **AI used for this answer:** No
- **If yes, how:** (e.g. brainstorming, grammar, concept explanation)
- **External references (links):** *(required when AI = Yes; encouraged otherwise)*

Write in English. Keep answers concise and specific.

Questions

Question 1: Data Filtering & Domain Relevance

Notebook: Section 3.2 — Question 1 (take home)

Task: Filter the dataframe **X** for the category **sci.med** and check its length.

Questions:

1. How many documents are in this specific category?
2. From a data mining perspective, why is isolating specific categories a crucial first step before applying algorithms like frequent pattern mining?

Answer:

1. **594**
2. Because frequent pattern mining finds items or words that repeatedly occur together. Without isolating specific categories, the data in question contains vocabulary from multiple domains. Hence, that creates noise and may produce misleading patterns

AI used for this answer: No

If yes, how:

External references (links): https://medium.com/@kurapatikarthik123/pattern-recognition-3ac6aeea00d2

Question 2: Handling Missing Data

Notebook: Section 4.1 — Question 2 (take home) (after the **dropna()** demonstration)

Task: In Section 4.1, a dummy record with a missing value is injected and then removed using the **dropna()** function.

Question: If 40% of documents were missing the **category_name** label, why would simply dropping these rows (with **dropna()**) be a flawed data mining strategy? Name one objective alternative so the text data can still be utilized.

Answer:

Dropping 40% of documents is a bad strategy because the missing data is big and the removal will significantly impact the analysis. If the labels are missing systematically, the pattern discovery of the model will be incomplete or misleading. A solution to this can be training a classifier on the 60% labeled data and using it to label the rest (semi-supervised learning)

AI used for this answer: Yes

If yes, how: To reformulate my ideas

External references (links):
https://medium.com/@adnan.mazraeh1993/certainly-212c21c25408

Question 3: Data Sampling Risks

Notebook: Section 5.1 — Question 3 (take home)

Task: In Section 5.1, the code uses **X_sample = X.sample(n=1000)** to create a subset. Compare the bar chart of the original category distribution to the sampled distribution.

Questions:

1. Do the exact category proportions perfectly match the original?
2. What is the specific risk of simple random sampling on a highly imbalanced dataset?

Answer:

1. **No**
2. On a highly imbalanced dataset, random sampling can remove completely or under-sample the minority class of the dataset (due to its nature of preserving the proportions of the original dataset), leading to poor prediction on critical rare events.

AI used for this answer: Yes

If yes, how: Yes, debate on the nature of pandas.DataFrame.sample

External references (links):
https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sample.html

---

Question 4: Text Vectorization and Matrix Sparsity

Notebook: Section 5.3 — Question 4 (take home)

Task: In Section 5.3, the **CountVectorizer** converts text into a document-term matrix using **stop_words='english'**. Change this parameter to **stop_words=None** and re-run the cell to check the shape of **X_counts**.

Questions:

1. Does the total number of features (columns) increase or decrease?
2. Since stopwords are highly common words that appear in almost all documents, what effect does adding them back have on the overall sparsity (the percentage of zeros) of the matrix?

Answer:

1. Increase from 35482 to 35788
2. It reduces the sparsity

AI used for this answer: No

If yes, how:

External references (links):

Question 5: Feature Selection — Variance Thresholding

Notebook: Section 5.3.1 — Question 5 (take home)

Task: In the variance filtering section, re-run **variance_filter** using threshold values **threshold = 0.0, threshold = 0.01, and threshold = 0.05**. Record the number of features marked as **keep == True**.

Questions:

1. What happens to the total size of the vocabulary as the variance threshold increases from 0.0 to 0.05?
2. In terms of document frequency, what kinds of words have variance close to zero and are eliminated first?
3. Why is eliminating near-zero variance features considered a safe way to reduce dimensionality without harming model accuracy?

Answer:

Threshold Number of terms kept (keep == True)

0.0 : 35482

0.01 : 5089

0.05 : 1491

1. Decrease significantly from 35482 to 1491
2. The common words in english such a, is, of that all sentences have and the very niche/rare word that almost no sentence has
3. Because words that appear in every sentence and words that rarely appear does not help us to establish a pattern hence they don't help to categorize. So since they don't help anything at all, we can safely consider them noise. And we can safely eliminate noise without harming model accuracy.

AI used for this answer: No

If yes, how:

External references (links):

Question 6: Feature Correlation — Linear (Pearson) vs. Monotonic (Spearman)

Notebook: Section 5.3.2 — Question 6 (take home)

Task: Run the correlation filter against the binary indicator for the **comp.graphics** class. Inspect the top 10 features sorted by **pearson_r** and compare them to the top 10 features sorted by **spearman_r**.

Questions:

1. Are the top-correlated words generic stop words or domain-specific terminology for graphics?
2. What is the mathematical difference in what **Pearson** correlation measures versus what **Spearman** rank correlation measures?
3. In what specific scenario would a word have a high **Spearman** rank correlation with a topic, but a significantly lower **Pearson** correlation?

Answer:

1. For Pearson, the top-correlated words are generic stop words. For Spearman, they are domain-specific words
2. Pearson correlation isn't fitted for classification problems (when the output is either 0 or 1) because it tries to explain the relationship between the amount of domain-specific words and the class related linearly. For example, when the document has 1 word related to its class and it actually is in the target class, its output is 1. But if the document has 10 words related to its class, it is still evaluated at 1. Hence when trying to draw a linear through all these points, it is possible that it misses out words that are repeated the most of the time. Spearman on the other hand, is created to deal with this problem by introducing a ranking system, making it resilient to extreme counts spike of related words
3. When a word appears consistently in related topic documents with small counts, but it appears in a few-off topic documents with huge counts, that will drag Pearson down. Spearman ignores the size of those spikes.

AI used for this answer: Yes

If yes, how: Used AI to review my understanding of these 2 concepts

External references (links):
https://support.minitab.com/en-us/minitab/help-and-how-to/statistics/basic-statistics/supporting-topics/correlation-and-covariance/a-comparison-of-the-pearson-and-spearman-correlation-methods/

Question 7: Frequent Pattern Mining — Algorithm & Parameter Dynamics

Notebook: Section 5.4.2 — Question 7 (take home)

Task: In the Frequent Pattern Mining section, run **Top-K FAE** with **k = 100 and k = 500**. Then, run **MaxFPGrowth** using minimum support thresholds **minSup=9 and minSup= 3**.

Questions:

1. How does decreasing **minSup** (from 9 to 3) in **MaxFPGrowth** affect the execution runtime and the total number of discovered patterns?
2. How does the parameter k in **Top-K FAE** differ fundamentally from **minSup** in **MaxFPGrowth** when deciding which patterns to keep?

3. Why are "maximal" frequent patterns structurally more concise than standard frequent patterns?

Answer (1) — Runtime and pattern count when lowering minSup: Both runtime and pattern count increase significantly

Answer (2) — Top-K k vs MaxFPGrowth minSup: Top-K FAE is rank-based (k = 100 means return the first 100 highest-ranked patterns). MaxFPGrowth is threshold-based (minSup=3 means return all patterns that was found in at least 3 documents)

Answer (3) — Maximal vs standard frequent patterns: Because the maximal frequent patterns are also the largest pattern that covers all of the smaller, subset patterns. Hence the maximal is more concise.

AI used for this answer: Yes

If yes, how: Explained to me how minSup differs from MaxFPGrowth

External references (links): https://link.springer.com/article/10.1007/s41019-024-00241-2

Question 8: Dimensionality Reduction — Motivation

Notebook: Section 5.5 — Question 8 (take home) (before PCA/t-SNE code)

Task: After text vectorization, the **X_counts** matrix creates tens of thousands of distinct features (words).

Questions:

1. If we want to visually map how these documents group by category on a scatter plot, why is it mathematically and visually impossible to plot the raw **X_counts** matrix directly?
2. What specific category of data mining algorithms (referenced in the helper files) must be applied first?

Answer:

1. Because X_counts represent every document using thousands of words. Each of these words is represented by an axe. Therefore, each document is a point in a space of thousands dimensions. Hence, it is impossible to draw this.
2. **Dimensionality reduction (visualization_helpers.py)**

AI used for this answer: No

If yes, how:

External references (links):

Question 9: Dimensionality Reduction — Linear (PCA) vs. Non-linear Manifolds (t-SNE/UMAP)

Notebook: Section 5.5 — Question 9 (take home) (after 2D/3D plots)

Task: In the 2D/3D Dimensionality Reduction section, compare the scatter plot projection generated by **PCA** against that generated by **t-SNE** or **UMAP**.

Questions:

1. Which method **(PCA or t-SNE/UMAP**) shows distinct, separated cluster islands for the 4 document categories, and which shows heavy overlap around the center?
2. Why does **PCA** struggle to separate high-dimensional text categories compared to **t-SNE** or **UMAP**?
3. Why must data analysts be careful not to over-interpret the physical distance between separate clusters in a **t-SNE** plot?

Answer:

1. PCA provides very little category separation. Most documents are compressed near the origin, while a few extreme documents stretch the axes. t-SNE reveals more category-specific regions, but considerable overlap remains near the center. UMAP gives the clearest partial separation, with visible branches dominated by different categories, although it still does not separate all four categories perfectly.
2. Because PCA works well for linear relationships. Since, documents and word features are non-linear relationships (as we already proved above) then PCA isn't a good fit here.
3. Because t-SNE mainly attempts to preserve local neighborhoods at the expense of global structure. Hence their positions and spacing aren't reliable

AI used for this answer: Yes

If yes, how: ask AI to verify my responses based on what i read in the below article

External references (links): https://biostatsquid.com/pca-umap-tsne-comparison/

Question 10: One-Hot Binarization

Notebook: Section 5.6 — Question 10 (take home)

Task: In Section 3.1, examine **X['category']** (integers 0, 1, 2, 3). Then look at the output of **LabelBinarizer** in Section 5.6, where each category is converted into a binary vector (e.g., [1, 0, 0, 0]).

Questions:

1. How many binary columns are created to represent the 4 categories?
2. If an algorithm (like linear regression or KNN) is given the raw integer column **(0, 1, 2, 3)** as input, what false mathematical assumption does it make about the categories?
3. How does one-hot binarization eliminate this false assumption?

Answer:

1. **4**
2. False assumption on distance : category 3 is closer to category 2 than category 0. False assumption on value : category 2 is bigger than category 0
3. They create 4 binary columns. So each category is like a vector. And in a 4D space, these 4 vectors are equally distant (which exactly how we want to perceive these 4 categories)

AI used for this answer: No

If yes, how:

External references (links):
